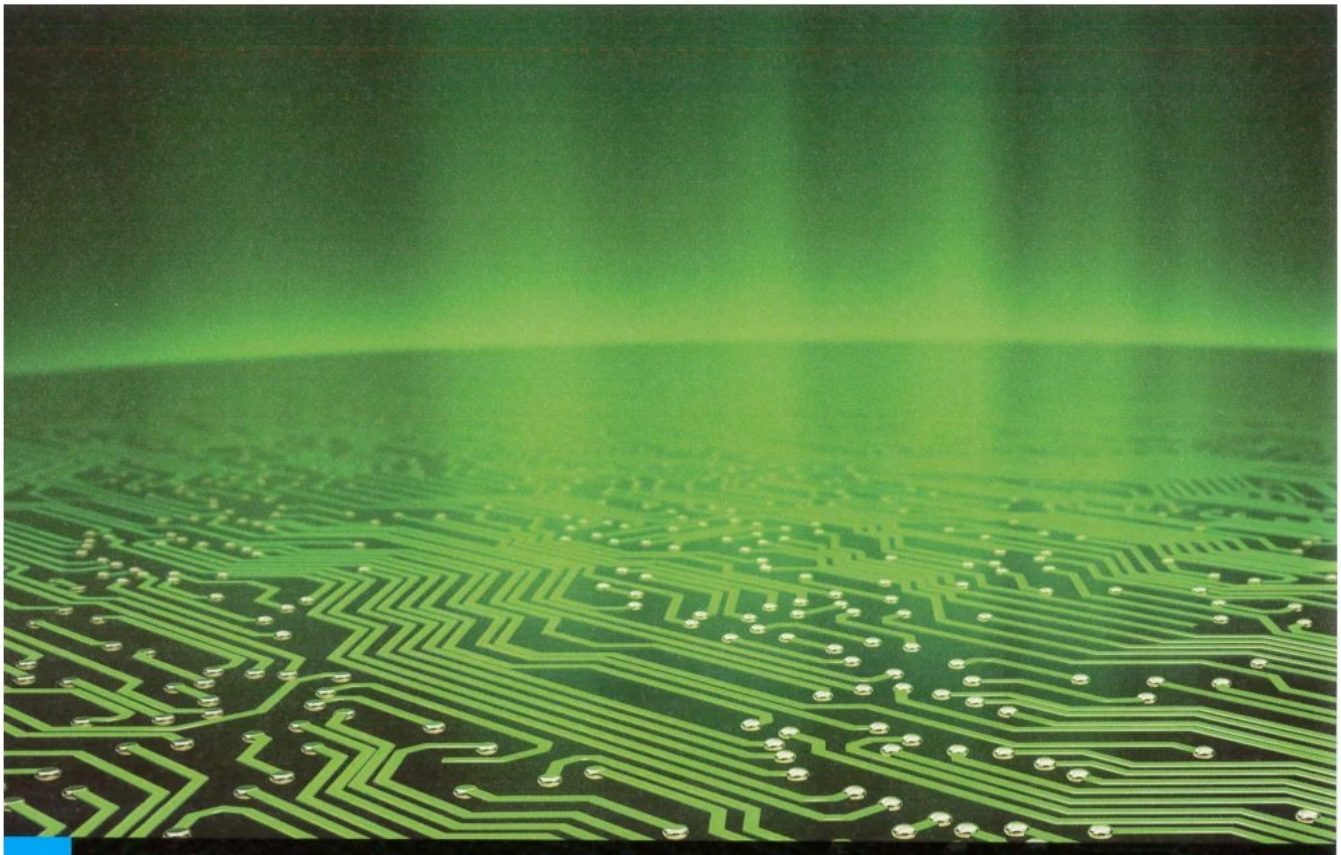
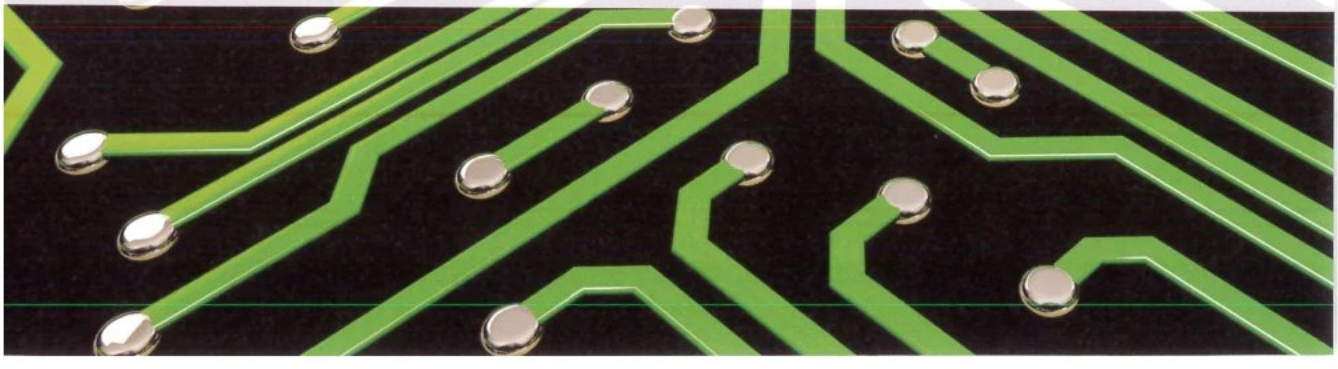


Chapter 18:
Hardware and virtual machines

Learning objectives

By the end of this chapter you should be able to:

- show understanding of Reduced Instruction Set Computers (RISC) and Complex Instruction Set Computers (CISC) processors
- show understanding of the importance/use of pipelining and registers in RISC processors
- show awareness of the four basic computer architectures: SISD, SIMD, MISD, MIMD
- show understanding of the characteristics of massively parallel computers
- show understanding of the concept of a virtual machine.



18.01 The control unit

While a program is being executed, the CPU is receiving a sequence of machine-code instructions. It is the responsibility of the control unit within the CPU to ensure that each machine instruction is handled correctly. There are two methods by which a control unit can be designed to allow it to perform its function.

One method is for the control unit to be constructed as a logic circuit. This is called the hard-wired solution. The machine-code instructions are handled directly by hardware.

The alternative method is for the control unit to use microprogramming. In this approach, the control unit contains a ROM component that stores the microinstructions or microcode for microprogramming, often referred to as firmware. In the next section we will see what might influence the choice of design of the control unit.

18.02 CISC and RISC processors

A processor will have an architecture which refers to its physical construction. However, a processor will also have what is termed an 'instruction set architecture'.

This is concerned with:

- the instruction set
- the instruction format
- the addressing modes
- the registers accessible by instructions.

The choice of the instruction set is the main factor in distinguishing one instruction set architecture from another.

In the early days of computing a significant factor in choosing the instruction set architecture for a computer was that it should make the writing of a compiler for a high-level language easier. At that time the term did not exist, but we would now refer to this as being the architecture for a **Complex Instruction Set Computer (CISC)** .

This philosophy began to be challenged in the late 1970s. It was argued that using a **Reduced Instruction Set Computer (RISC)** would be a better approach. Table 18.01 contains a number of features that distinguish RISC from CISC.



KEY TERMS

Complex Instruction Set Computer (CISC): a single instruction can be more complex and involve more loading of data from memory

Reduced Instruction Set Computer (RISC): a single instruction is simpler, requiring minimal loading of data from memory

RISC	CISC
Fewer instructions	More instructions
Simpler instructions	More complex instructions
Small number of instruction formats	Many instruction formats
Single-cycle instructions whenever possible	Multi-cycle instructions
Fixed-length instructions	Variable-length instructions
Only load and store instructions to address memory	Many types of instructions to address memory
Fewer addressing modes	More addressing modes
Multiple register sets	Fewer registers
Hard-wired control unit	Microprogrammed control unit
Pipelining easier	Pipelining more difficult

Table 18.01 Comparison of RISC with CISC

The following are some points to note.

- For RISC the term 'reduced' affects more than just the number of instructions.
- A reduction in the number of instructions is not the major driving force for the use of RISC.
- The reduction in the complexity of the instructions is a key feature of RISC.
- The typical CISC architecture contains many specialised instructions.
- The specialised instructions are designed to match the requirement of a high-level programming language.
- The specialised instructions require multiple memory accesses which are very slow compared with register accesses.
- The simplicity of the instructions for a RISC processor allows data to be stored in registers and manipulated in them with no resource to memory access other than that necessary for initial loading and possible final storing.
- The simplicity of RISC instructions makes it easier to use hard-wiring inside the control unit.
- The complexity of many of the CISC instructions makes hard-wiring much more difficult so microprogramming is the norm.

Extension Question 18.01

Can you find out whether the processors in any systems you are using are described as RISC or CISC?

Pipelining

One of the major driving forces for creating RISC processors was the opportunity they would provide for efficient **pipelining**. Pipelining is a form of parallelism applied specifically to instruction execution. Other forms of parallelism are discussed in Section 18.03.



KEY TERM

Pipelining: instruction-level parallelism

The underlying principle of pipelining is that the fetch–decode–execute cycle described in Chapter 5 (Section 5.06) can be separated into a number of stages. One possibility is a five-stage model consisting of:

		Clock cycles						
		1	2	3	4	5	6	7
Processor units	Instruction fetch (IF)	1.1	2.1	3.1	4.1	5.1	6.1	7.1
	Instruction decode (ID)		1.2	2.2	3.2	4.2	5.2	6.2
	Operand fetch (OF)			1.3	2.3	3.3	4.3	5.3
	Instruction execute (IE)				1.4	2.4	3.4	4.4
	Result write back (WB)					1.5	2.5	3.5

Figure 18.01 Pipelining for five-stage instruction handling

Figure 18.01 shows how pipelining would work with this five-stage breakdown of instruction handling. For pipelining to be implemented, the construction of the processor must have five independent units, with each handling one of the five stages identified. This explains the need for a RISC processor to have many register sets; each processor unit must have access to its own set of registers. Figure 18.01 uses the representation 1.1, 1.2 and so on to define the instruction and the stage of the instruction. Initially only the first stage of the first instruction has entered the pipeline. At clock cycle 6 the first instruction has left the pipeline, the last stage of instruction 2 is being handled and instruction 6 has just entered.

Once under way, the pipeline is handling five stages of five individual instructions. In particular, at each clock cycle the complete processing of one instruction has finished. Without the pipeline the processing time would be five times longer.

One issue with a pipelined processor is interrupt handling. The discussion in Chapter 5 (Section 5.08) referred to a processor with instructions handled sequentially. This approach where a check for any interrupt is made following the execution of an instruction is applicable to a CISC processor. It would also be applicable to a RISC processor if there were no pipelining. However, this is an unlikely circumstance. In the pipelined system described above there will be five instructions in the pipeline when an interrupt occurs. One option for handling the interrupt is to erase the pipeline contents for the latest four instructions to have entered. Then the normal interrupt-handling routine can be applied to the remaining instruction. The other option is to construct the individual units in the processor with individual program counter registers. This option allows current data to be stored for all of the instructions in the pipeline while the interrupt is handled.

Discussion Point:

Consider the two consecutive instructions:

```
ADD R1, R2, R3

ADD R5, R1, R4
```

These are typical three-register instructions favoured for RISC. The first adds the contents of registers R2 and R3 and stores the result in R1. The next instruction is similar but uses

the value stored in R1. In a pipelined structure, the second instruction will be reading the contents of R1 before the previous instruction has placed the value there. How could this potential problem be overcome?

18.03 The basic computer architectures

A well-established approach to describing different computer architectures is to consider the number of instruction streams and the number of data streams. This leads to four different types.

Single Instruction Stream Single Data Stream (SISD)

SISD is the typical arrangement found in early computers which was also adopted for the earliest microprocessors. The functioning is purely sequential with no parallelism. Figure 18.02 illustrates how individual elements in an array are manipulated using SISD.

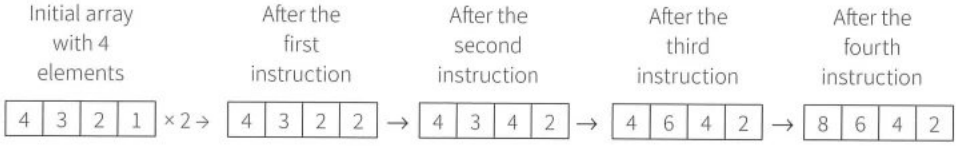


Figure 18.02 SISD sequential processing of array elements

The same instruction is used repeatedly to carry out the multiplication by two.

Single Instruction Stream Multiple Data Stream (SIMD)

SIMD can be considered as parallelism applied to the data stream. The difference when array elements are processed is shown in Figure 18.03.

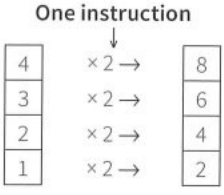


Figure 18.03 SIMD parallel processing of array elements

The architecture for SIMD is usually represented by a diagram such as Figure 18.04.

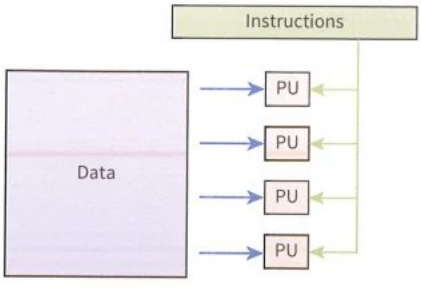


Figure 18.04 Schematic representation of the SIMD architecture

In Figure 18.04 the structure shows four data streams entering four individual components which are simultaneously supplied with the same instruction. The components are labelled PU (Processing Unit). Sometimes the components are labelled PE (Processing Element). Whatever name is used, the components are arithmetic logic units. One of the reasons for the different names is that there are options for how this architecture could be implemented.

TIP

Be careful to distinguish between the pipelining of instructions where different instructions are executed in parallel and SIMD when only one instruction is being simultaneously executed on different data streams.

One option has been used in computers that are called array or vector processors. These might have a parallel set of registers; one for each data stream. Alternatively, there would be a large register, perhaps with 64 or 128 bits which could store four data values at the same time. In this type of implementation, the parallelism is built into just one processor.

The alternative is the multi-core processor where four individual processors work in parallel. In this case each processor is likely to have its own dedicated cache memory to provide the data stream.

KEY TERMS

SISD: Single Instruction Stream Single Data stream; a single processor accessing one memory

SIMD: Single Instruction Stream Multiple Data stream; processing of parallel data input requiring one control unit instructing multiple processing units

MISD: Multiple Instruction Stream Single Data stream; does not exist in a single architecture

MIMD: Multiple Instruction Stream Multiple Data stream; multiple processors asynchronously processing parallel data input

Multiple Instruction Stream Single Data Stream (MISD)

MISD is not evidenced in any individual computer architecture design. One example where the approach could be implemented would be in a fault-tolerant system. The same data stream could be fed into two or more processors. The output would only be accepted if the same output was produced by all of the processors.

Multiple Instruction Stream Multiple Data Stream (MIMD)

The **MIMD** architecture is similar to that for the type of SIMD architecture shown in Figure 18.04 with more than one processing unit receiving the parallel data streams. The difference is that each Processing unit does not execute the same instruction. The multiple data stream can be provided by a suitably partitioned single memory. Each Processing Unit might have a dedicated cache memory.

Massively parallel computer systems

The **MIMD** architecture can be implemented in multicomputer systems known as massively parallel computers. These are the systems used by large organisations for computations involving highly complex mathematical processing. They are the latest type of 'supercomputer'. The major difference in architecture is that instead of having a bus structure

to support multiple processors there is a network infrastructure to support multiple computer units. The programs running on the different computers can communicate by passing messages using the network.

An alternative type of multicomputer system is cluster computing using PCs (sometimes referred to as a 'server farm').

Whatever the technology used, these systems have an extremely large number of individual processors working in parallel.

Extension Question 18.02

The IBM Sequoia and a Beowulf cluster are different examples of cluster computing. You might wish to carry out some research to get some details of their computing power.

18.04 Virtual machines

A virtual machine is software not hardware. The most usual type of virtual machine is the **system virtual machine** which is software that emulates the hardware of a real computer system.

KEY TERM

System virtual machine: the emulation of computer system hardware using software

Remember that when a virtual machine is not being used an application program requires support from an operating system in order for the program to run on the hardware. The principle of a virtual machine is that a process interacts directly with a software interface provided by an operating system. The logical structure for the operation of a system virtual machine is shown in Figure 18.05.

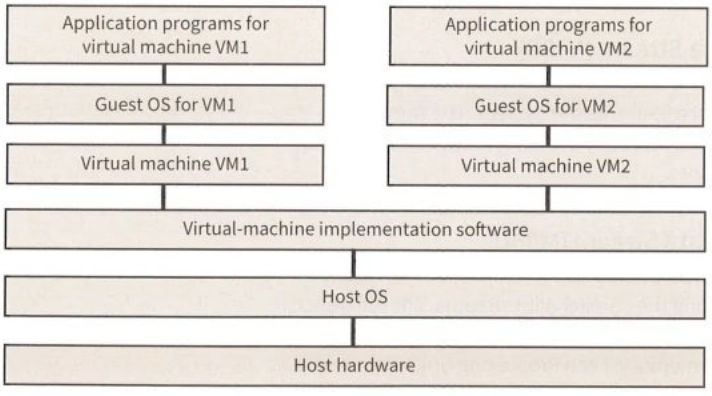


Figure 18.05 Logical structure for a system virtual machine implementation

The following points should be noted when looking at Figure 18.05.

1 The application programs are installed with the assistance of a guest OS. This guest OS will support the running application by interacting with the virtual machine as though it were the hardware that the guest OS would normally run on.

2 The virtual machine implementation software can be considered to be a utility program which, when running, is supported by the particular host OS which is specific to the host hardware.

3 There can be application programs running at the same time directly on the host hardware under the control of the host OS.

The main advantage of the virtual machine approach is that more than one different operating system can be made available on one computer system. This is particularly valuable if an organisation has legacy systems and wishes to continue to use the old software but does not wish to keep the old hardware. Alternatively, the same operating system can be made available many times by companies with large mainframe computers that offer server consolidation facilities. Different companies can be offered their own virtual machine running as a server.

One drawback to using a virtual machine is the time and effort required for implementation. Another drawback is the fact that the implementation will not offer the same level of performance that would be obtained on a normal system.

Discussion Point:

Are there other advantages or disadvantages in using a system virtual machine?

The Java virtual machine discussed in Chapter 8 (Section 8.05) is an example of a process virtual machine, based on a different underlying concept. The process virtual machine provides a platform-independent programming environment that allows a program to execute in the same way on any platform. This is specific software that only supports running a Java program. A system virtual machine supports any application.

Reflection Point:

The description 'virtual' occurs in several places in this book. Are you clear about how these different examples have their own specific context and meaning?

Summary

- A control unit can be hard-wired or microprogrammed.
- RISC (Reduced Instruction Set Computers) processors have a number of advantages compared to CISC (Complex Instruction Set Computers).
- Pipelining is one of the reasons for choosing a RISC architecture.
- Parallelism can be based at the instruction level, processor level or computer level.
- A system virtual machine is software emulating hardware.
- The Java Virtual Machine is an example of a process virtual machine.

Exam-style Questions

1 a Computer systems are now often constructed with RISC processors.

i State what the acronym RISC stands for. [1]

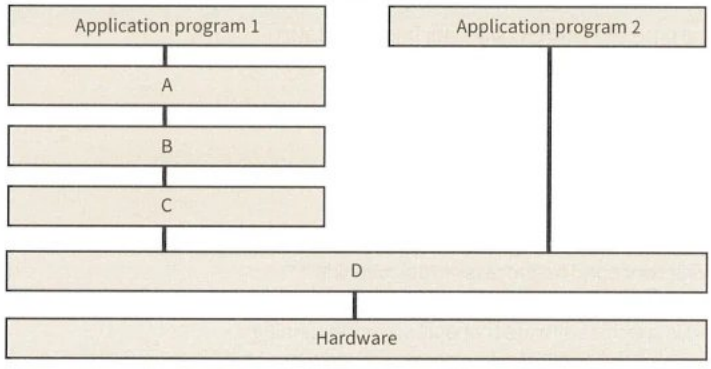
ii State **four** characteristics to be expected of a RISC system. [4]

b A RISC processor is likely to be 'hard-wired'.

i Explain what this term means and which specific part of the processor will be hard-wired. [3]

ii State what the alternative to hard-wiring is and what hardware component is needed to be part of the processor to allow this alternative to be implemented. [2]

2 The following diagram represents a system which has implemented a virtual machine.



a For each of A, B, C, D give a suitable name. [4]

b For each of A, B, C and D give a brief description of the function of the feature. [8]

c Explain why the diagram is different for Application 2 [2]

3 a Parallelism can be achieved in a number of ways.

i Identify **three** different types of parallelism. [3]

ii Identify which type pipelining belongs to. [1]

iii Using a diagram, explain how pipelining works. [5]

b Interrupt handling is not so straightforward in a pipelined system. Explain why this is so and give a brief account of how problems can be avoided. [3]

4 a Three descriptions and two types of processor are shown below.

Draw a line to connect each description to the appropriate type of processor.

Description	Type of processor
Makes extensive use of general purpose registers	RISC
Many addressing modes are available	CISC
Has a simplified set of instructions	

[3]

b In a RISC processor three instructions (A followed by B, followed by C) are processed using pipelining.

The following table shows the five stages that occur when instructions are fetched and executed.

i The "A" in the table indicates that instruction A has been fetched in time interval 1.

Complete the table to show the time interval in which each stage of each instruction (A, B, C) is carried out.

Stage	Time interval								
	1	2	3	4	5	6	7	8	9
Fetch instruction	A								
Decode instruction									
Execute instruction									
Access operand in memory									
Write result to register									

[3]

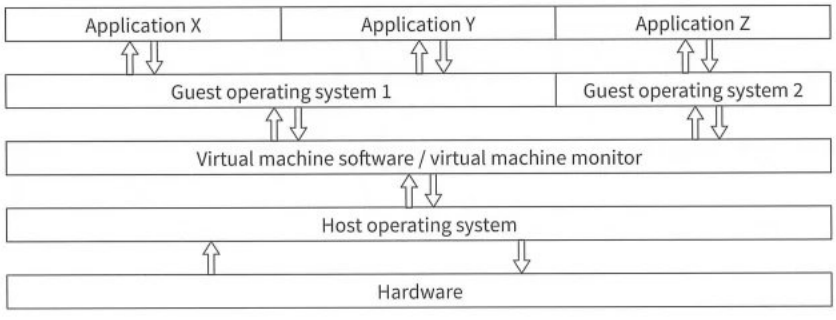
ii The completed table shows how pipelining allows instructions to be carried out more rapidly. Each time interval represents one clock cycle.

Calculate how many clock cycles are saved by the use of pipelining in the above example.

Show your working. [3]

Cambridge International AS & A level Computer Science 9608 paper 31 Q4 November 2015

5 a The following diagram shows how applications X, Y and Z can run on a virtual machine system.



i The virtual machine software undertakes many tasks.

Describe **two** of these tasks. [2]

ii Explain the difference between a guest operating system and a host operating system. [2]

b A company uses a computer as web server. The manufacturer will no longer support the computer's operating system (OS) in six months time. The company will then need to decide on a replacement OS.

The company is also considering changing the web server software when the OS is changed.

Whenever any changes are made, it is important that the web server service is not disrupted.

In developing these changes, the company could use virtual machines.

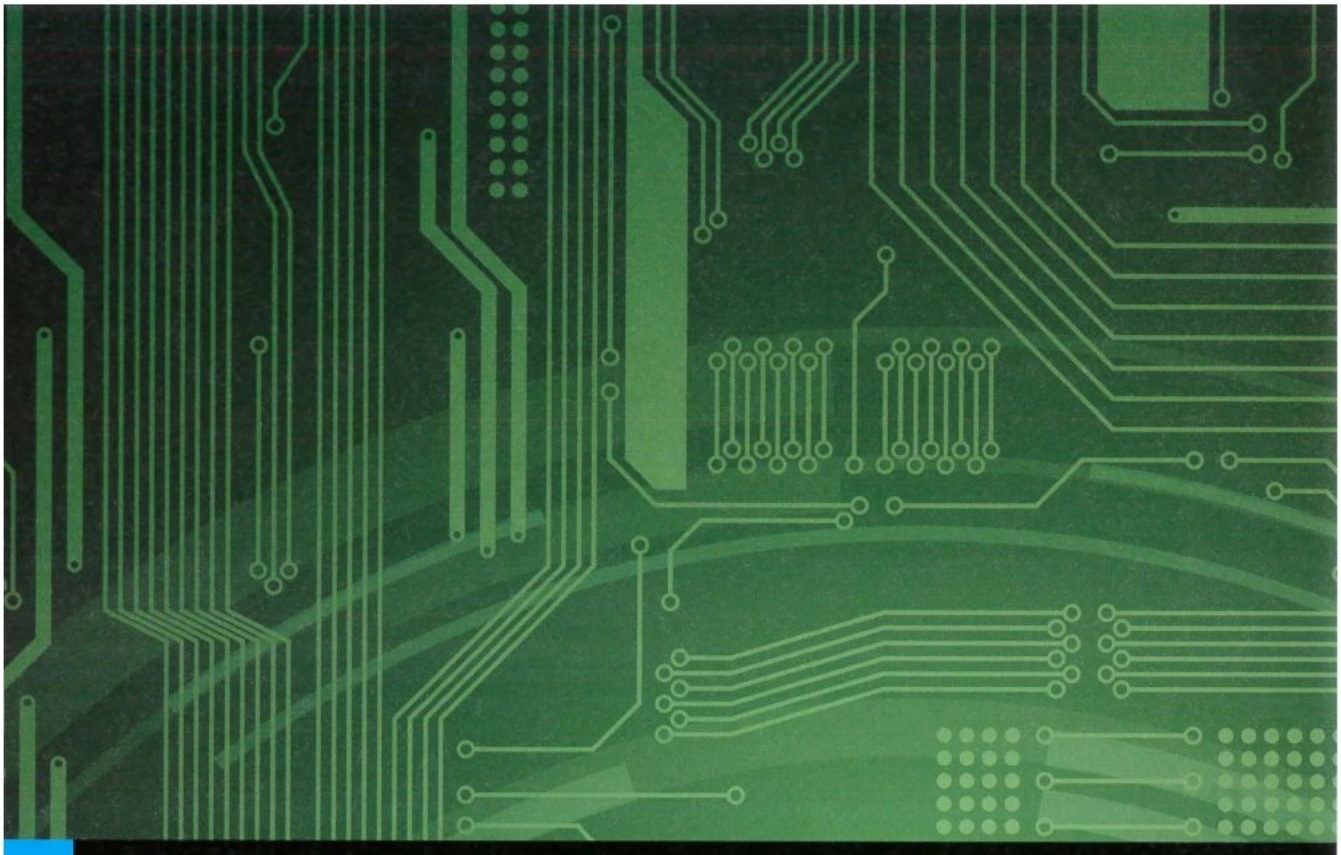
i Describe **two** possible uses of virtual machines by the company. [4]

The web server often has to handle many simultaneous requests.

ii The company uses a virtual machine to test possible solutions to the changes that they will need to make.

Explain **one** limitation of this approach. [2]

Cambridge international AS & A Level Computer Science 9608 paper 31 Q3 June 2016

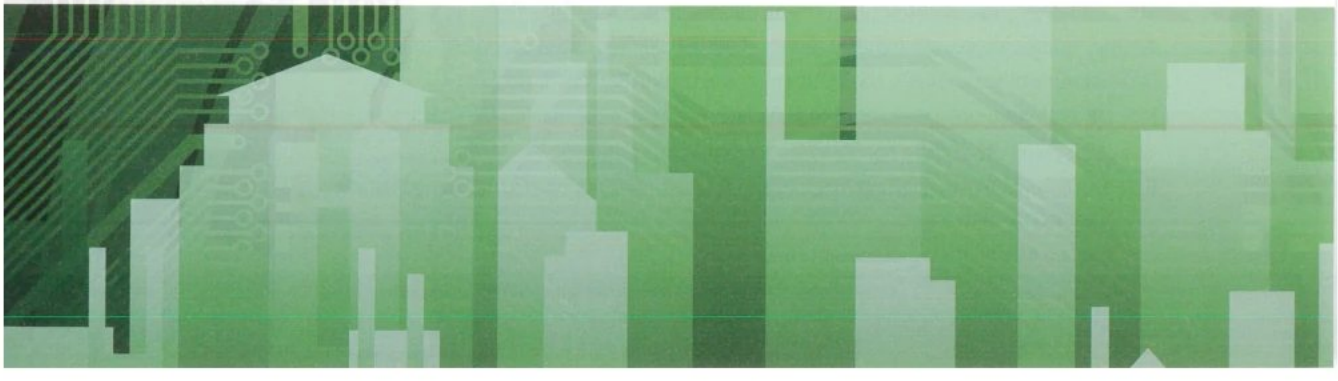


Chapter 19:
Logic circuits and Boolean algebra

Learning objectives

By the end of this chapter you should be able to:

- produce truth tables for logic circuits including half adders and full adders
- show understanding of a flip-flop (SR, JK)
- show understanding of Boolean algebra
- show understanding of Karnaugh Maps.



19.01 Logic circuits

Chapter 4 introduced the symbols for logic gates that are used in logic circuits and discussed the relationships between logic circuits, truth tables and logic expressions. This chapter introduces some specific circuits that are used to construct components that provide functionality in computer hardware.

The half adder

A fundamental operation in computing is binary addition. The result of adding two bits is either 1 or 0. However, when 1 is added to 1 the result is 0 but there is a carry bit equal to 1. This cannot be ignored if two numbers with several bits in each are being added.

The simplest circuit that can be used for binary addition is the **half adder**. This can be represented by the diagram in Figure 19.01. The circuit takes two input bits and outputs a sum bit (S) and a carry bit (C).

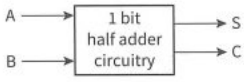


Figure 19.01 A half adder

The truth table for this circuit is shown in Table 19.01.

Input		Output	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 19.01 The truth table for a half adder

One possible circuit can be defined directly by examination of the truth table. It can be seen that the only combination of inputs that produces a 1 for the carry bit is when two 1 bits are input. The truth table for the C output is in fact the AND truth table. The truth table for the S output can be seen to match that for the XOR operator which is shown in Figure 4.02 in Chapter 4 (Section 4.04). Therefore, one circuit that would produce the half adder functionality would contain an AND gate and an XOR gate with each gate receiving input from A and B.

This is only one of several circuits that would provide the functionality. The NAND and NOR gates are universal gates. Any logic circuit can be constructed using only NAND gates or only NOR gates. This fact combined with the ease of manufacture of these gates leads circuit manufacturers to prefer their use. The circuit shown in Figure 19.02 consisting only of NAND gates has the correct logic to produce the C and S outputs and is a likely choice for implementation.

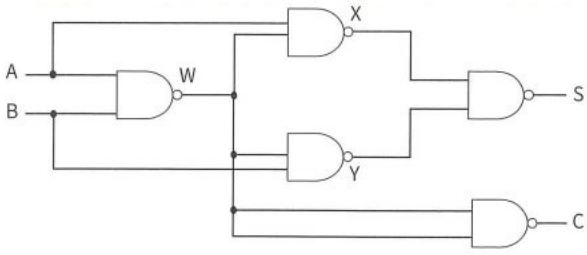


Figure 19.02 A half adder circuit constructed from NAND gates

Question 19.01

In Figure 19.02, can you identify the individual circuits that represent the AND operator and the XOR operator?

TASK 19.01

Use the intermediate points labelled W, X and Y to construct a truth table for the circuit shown in Figure 19.02. Check that this reproduces the truth table shown as Table 19.01.

The full adder

If two numbers expressed in binary with several bits to be added, the addition must start with the two least significant bits and then proceed to the most significant bits. At each stage, the carry from the previous addition has to be incorporated into the current addition. If a half adder is used each time, there has to be separate circuitry to handle the carry bit because the half adder only takes two inputs.

The **full adder** is a circuit that has three inputs including the previous carry bit. The truth table is shown as Table 19.02.

Input			Output	
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 19.02 The truth table for a full adder

One possible circuit for implementation contains two half adder circuits and an OR gate as shown in Figure 19.03.

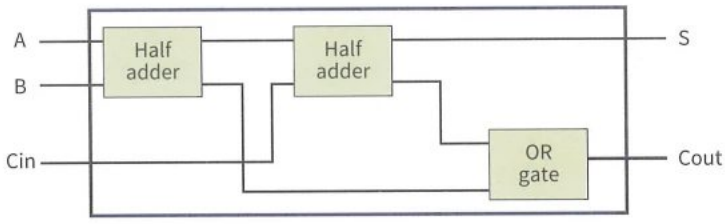


Figure 19.03 A possible implementation of a full adder

KEY TERMS

Half adder circuit: a circuit which performs binary addition of two individual bits

Full adder circuit: a circuit which performs binary addition of two individual bits and an input carry bit

As before, it is possible to construct the circuit entirely from NAND gates as shown in Figure 19.04.

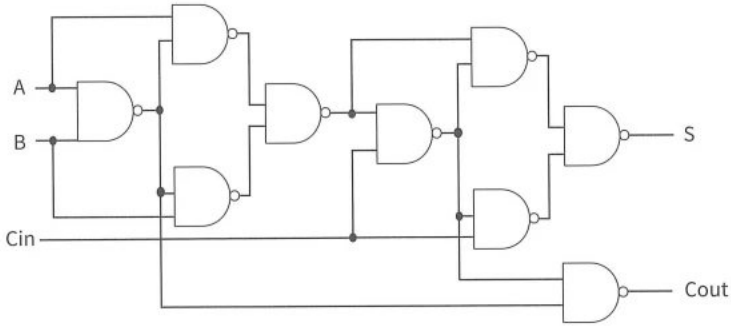


Figure 19.04 A full adder circuit constructed from NAND gates

Discussion Point:

Can you see how full adders could be combined to handle addition of, for example, four-bit binary numbers? What happens to the carry input for the first addition?

19.02 Sequential logic circuits

All of the circuits so far encountered in this book have been **combinational circuits**. For such a circuit the output is dependent only on the input values. An alternative type of circuit is a **sequential circuit** where the output depends on the input and on the previous output.

KEY TERMS

Combinational circuit: a circuit in which the output is dependent only on the input values

Sequential circuit: a circuit in which the output depends on the input values and the previous output

The SR flip-flop

The SR flip-flop or 'latch' is a simple example of a sequential circuit. It can be constructed with two NAND gates or two NOR gates. Figure 19.05 shows the version with two NOR gates

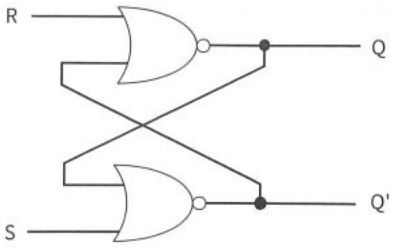


Figure 19.05 A circuit for an SR flip-flop using NOR gates

The cross-coupled circuit with output feedback has some interesting consequences. For example, if we consider the hypothetical situation where R, S, Q and Q' all had value 0 we can see that this would be self-contradictory because two inputs value 0 to a NOR gate produce a 1 output. A state with Q and Q' both set to value 1 would be self-contradictory because the output from a NOR gate is always 0 if either input is 1. The flip-flop is therefore a two-state device. Either it has Q=1 and Q'=0 or it has the reverse.

The truth table for the circuit can be presented as shown in Table 19.03. The two lines of the truth table where both S and R are input as 0 represent the condition when no signal is input to the flip-flop. If we consider the state with Q=1 and Q'=0:

- and we consider the condition that both S and R inputs are 0
- then the top NOR gate has inputs both 0
- giving output 1
- and the bottom NOR gate has inputs 1 and 0
- giving output 0.

This means that this is a self-consistent state which is referred to as the set state. A similar argument can be applied for the alternative to the set state which is the unset state with Q=0 and Q'=1.

The S and R inputs are for set and reset, respectively. The truth table shows that an input combination of S=0 and R=1 converts a set state to an unset state and an input combination of S=1 and R=0 converts an unset state to a set state.

Input signals		Initial state		Final state	
S	R	Q	Q'	Q	Q'
0	0	1	0	1	0
1	0	1	0	1	0
0	1	1	0	0	1
0	0	0	1	0	1
1	0	0	1	1	0
0	1	0	1	0	1

Table 19.03 A representation of a truth table for an SR flip-flop

These properties explain why the SR flip-flop can be used as a storage device for 1 bit and therefore could be used as a component in RAM because a value is stored but can be altered. The truth table does not contain rows for R=1 and S=1 because this leads to an invalid

state with both Q and Q' having value 0. Because of this the circuit must be protected from receiving an input signal on R and S simultaneously.

The alternative NAND gate circuit for the SR flip-flop has a similar structure but the labeling is different. The important difference is that setting is achieved with S=0 and R=1 and resetting with R=0 and S=1.

TASK 19.02

Find a diagram for a NAND gate SR flip-flop and construct the truth table.

The JK flip-flop

In addition to the possibility of entering an invalid state there is also the potential for a circuit to arrive in an uncertain state if inputs do not arrive quite at the same time. In order to prevent this, a circuit may include a clock pulse input to give a better chance of synchronising inputs. The JK flip-flop is an example.

The JK flip-flop can be illustrated by the symbol shown in Figure 19.06(a). A possible circuit is shown in Figure 19.06(b).

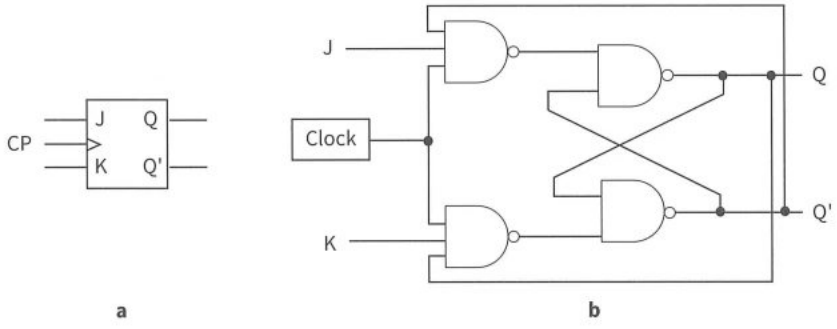


Figure 19.06 (a) A symbol for a JK flip-flop and (b) a possible circuit

TIP

There is one special case for a NAND gate; when all inputs are 1 the output is 0. There is one special case for a NOR gate; when all inputs are 0 the output is 1.

To understand the workings of the circuit you must first remember that a NAND gate gives a 1 output unless all inputs are 1. If the circuit is in the unset state with Q = 0 and Q' =1 this is stable and self-consistent as shown by the following argument:

- the clock and R both input 0
- the top-left NAND gate therefore has output 1
- this then leads to two 1 inputs to the top-right NAND gate
- ensuring that Q = 0.

If the input from J now becomes 1 and the clock pulse switches to a 1 then:

- the top-left NAND gate has all 1 inputs
- so the output is 0
- this causes the top-right NAND gate to output a 1 for Q

- there are now two 1 inputs to the bottom-right NAND gate
- so the Q' output becomes 0.

This shows how the J input is a set input. A similar argument shows that the K is a clear input. In this respect the JK flip-flop behaves in a similar way to the SR flip-flop as a storage device for one bit. However, there is an important difference in that if both J and K are input as a 1 then the values for Q and Q' are toggled (they switch value). This makes the JK flip-flop a more reliable device because there is no combination of input states that leave uncertainty as to which values are stored. The significant part of the truth table is shown as Table 19.04.

J	K	Clock	Q
0	0	↑	Q unchanged
1	0	↑	1
0	1	↑	0
1	1	↑	Q toggles

Table 19.04 Part of the truth table for a JK flip-flop

Question 19.02

Can you follow the logic in the JK flip-flop circuit to see that the truth table is an accurate representation of the logic for all combinations of input?

19.03 Boolean algebra basics

Chapter 4 (Section 4.01) introduced logic expressions consisting of logic propositions combined using Boolean operators. Boolean algebra provides a simplified way of writing a logic expression and a set of rules for manipulating an expression.

Whenever a form of algebra is used it is vital that there is an understanding of its meaning. As a simple example we can consider the following four interpretations of the meaning of 1 + 1:

```
1 + 1 = 2
1 + 1 = 10
1 + 1 = 0
1 + 1 = 1
```

The first shows denary arithmetic, the second binary arithmetic and the third bit arithmetic. The last one applies if Boolean algebra is being used. This is because in Boolean algebra 1 represents TRUE, 0 represents FALSE, and + represents OR. Therefore the fourth statement represents the logic statement:

TRUE OR TRUE is TRUE

There are options for the representation of Boolean algebra. For example, the symbols for AND and OR are sometimes represented as ∧ and ∨. There is also the option of writing A.B or AB for A AND B. The dot notation is used in this book. Finally, there are options for how NOT A (the inverse of A) can be represented. $\bar{A}$ is used here.

TIP

You do not need to remember the ∧ and ∨ notation.

Having established the notation for Boolean algebra we have to consider the rules that apply. These can formally be described as 'laws' or 'identities'. Table 19.05 contains a full listing.

Identity/Law	AND form	OR form
Identity	$1.A = A$	$0 + A = A$
Null	$0.A = 0$	$1 + A = 1$
Idempotent	$A.A = A$	$A + A = A$
Inverse	$A.\overline{A} = 0$	$A + \overline{A} = 1$
Commutative	$A.B = B.A$	$A + B = B + A$
Associative	$(A.B).C = A.(B.C)$	$(A + B) + C = A + (B + C)$
Distributive	$A + B.C = (A + B).(A + C)$	$A.(B + C) = A.B + A.C$
Absorption	$A.(A + B) = A$	$A + A.B = A$
De Morgan's	$\overline{A.B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}.\overline{B}$
Double Complement	$\overline{\overline{A}} = A$	

Table 19.05 Boolean algebra identities (laws)

Some of the names used for the identities may be unfamiliar to you. This is not a concern. You should note that for all but one of the identities there is an AND form and an OR form. Furthermore, it is important to note that an identity written in one form can be transformed into the other by interchanging each 0 or 1 and each AND or OR. For example, $0.A = 0$ which reads FALSE AND A is FALSE transforms into TRUE OR A is TRUE, written in the algebra as $1 + A = 1$.

It can also be seen that some of the identities look like those applying in normal algebra with AND functioning as multiplication and OR functioning as addition. So you can use the terms 'product' and 'sum' in the context of Boolean algebra.

TASK 19.03

It is vital that you can interpret a Boolean expression correctly. Go through Table 19.05 item by item and in each case read out the full meaning. For example:

$1 + A = 1$ can be read as 'one plus A equals 1'

but must be understood as 'TRUE OR A is TRUE'.

Although De Morgan's laws look complicated at first glance, they can be rationalised easily. The inverse of a Boolean product becomes the sum of the inverses of the individual values in the product. The inverse of a Boolean sum is the product of the individual inverses.

Unfortunately, using the algebra to simplify expressions is not something which can be learnt as a routine. It requires lateral thinking as Worked Example 19.01 will show.

WORKED EXAMPLE 19.01

Using Boolean algebra to simplify an expression

Let's consider a simple example:

$A + \overline{A}.B$ can be simplified to $A + B$

In order to simplify the expression, we have to first make it more complicated! This is where the lateral thinking comes in. The OR form of the absorption identity is $A + A.B = A$. This can be used in reverse to replace A by $A + A.B$ to produce the following:

$$A + A.B + \overline{A}.B$$

We can for the moment ignore the A.

Applying the AND form of the commutative law and the OR form of the distributive law in reverse we can see that:

$A.B + \overline{A}.B$ is the same as $B.A + B.\overline{A}$ which converts to $B.(A + \overline{A})$

This allows us to use the OR form of the inverse identity which converts $A + \overline{A}$ into 1. As a result, the full expression, including the A that was ignored, has become:

$$A + B.1$$

The AND form of the identity law can now be applied to the B.1 term to change it to B so the expression then becomes $A + B$.

19.04 Boolean algebra applications

Creating a Boolean algebra expression directly from a truth table

One formal approach to creating a Boolean algebra expression for a particular problem is to start with the truth table and apply the sum of products method. This establishes a 'minterm' for each row of the table that results in a 1 for the output. This can be illustrated using the half adder circuit truth table shown in Figure 19.01. The only row of the table creating a 1 output for C has a 1 input for A and for B. The product becomes A.B and the sum of products has only this one term so we have:

$$C = A.B$$

For the S output, there are two rows that produce a 1 output so there is a sum of products containing two minterms:

$$S = \overline{A}.B + A.\overline{B}$$

Note that the 0 in a row is represented by the inverse of the input symbol.

In certain cases Boolean algebra laws can be applied directly to parts of the truth table. For example, consider a truth table which contains the following two rows:

A	B	C	X
0	1	0	1
0	1	1	1

The output for X is the same whether C = 0 or C = 1. There must therefore be a minterm which has no reference to C which in this case will be $\overline{A}.B$.

The Boolean algebra representation of a logic circuit

This approach can also be used as part of the process of creating a Boolean algebra logic expression from a circuit diagram. The truth tables for the individual logic gates are used and then some algebraic simplification is applied.

WORKED EXAMPLE 19.02

Creating a Boolean algebra logic expression for a half adder circuit

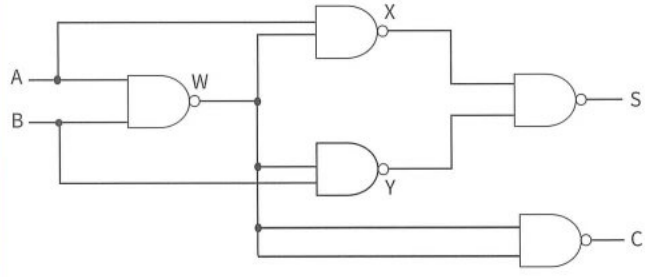


Figure 19.07 A half adder circuit

Figure 19.07 shows inputs A and B to a NAND gate with output W. The first three rows of the NAND truth table produce a 1 output so the sum of products has three minterms:

$$W = \overline{A}.\overline{B} + \overline{A}.B + A.\overline{B}$$

We can now consider the input of W to a NAND gate with A as the other input to produce the X output. Instead of using the NAND truth table we will use the fact that the NAND gate operates as an AND gate followed by a NOT gate.

The result of the AND operation is the product of the inputs so we get the following expression as output:

$$A.(\overline{A}.\overline{B} + \overline{A}.B + A.\overline{B})$$

This can be simplified by applying the distributive and inverse laws to give:

$$0 + 0 + A.A.\overline{B} \text{ which is simply } A.\overline{B}$$

To complete the NAND operation and get the value for X we have to apply the NOT operation which means we have to take the inverse of the above expression.

This is where we need the AND version of De Morgan's law which transforms the $A.\overline{B}$ into $\overline{A} + B$. So we have:

$$X = \overline{A} + B$$

The same laws applied to the output Y from the other intermediate NAND gate to give

$$Y = A + \overline{B}$$

Finally, we need to consider $A + \overline{B}$ and $\overline{A} + B$ being input to the final NAND gate. Again we can consider the AND operation first as the product of the inputs to produce the expression:

$$(A + \overline{B}).(\overline{A} + B)$$

We will not multiply this out; we will instead apply De Morgan's law directly to the expression to perform the inverse operation to complete the NAND operation. This gives:

$$S = \overline{A}.B + A.\overline{B}$$

This is the value obtained directly from the truth table so the algebra has been used correctly.

Extension Question 19.01

Worked Example 19.02 did not show that the circuit produced the correct output for C. Also a shortcut was used to reach the final form of S. Can you use Boolean algebra to find the form of C from the circuit and can you convert the expression for S if you start by using the distributive law before applying De Morgan's law?

19.05 Karnaugh maps (K-maps)

A Karnaugh map is a method of creating a Boolean algebra expression from a truth table. A K-map can make the process much easier than if you use sum-of-products to create minterms. If applied correctly a K-map produces the simplest possible form for the Boolean algebra expression.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Table 19.06 The truth table for the OR operand

The truth table for an OR gate, shown as Table 19.06, can be used to illustrate the method. The sum of products method produces the following expression:

$$X = \bar{A}.B + A.\bar{B} + A.B$$

This is not instantly recognisable as A + B but, with a little effort, using Boolean algebra laws it could be shown to be the same.

The Karnaugh map approach is simpler. The corresponding K-map is shown in Figure 19.08. Each cell in a Karnaugh map shows the value of the output X for a combination of input values for A and B.

	$\bar{B}$	B
$\bar{A}$	0	1
A	1	1

Figure 19.08 A K-map of the truth table in Table 19.06

The interpretation of a Karnaugh map follows these rules.

- Only cells containing a 1 are considered.
- Groups of cells containing 1s are identified where possible, with a group being a row, a column or a rectangle.
- Groups must contain 2, 4, 8 and so on cells.
- Each group should be as large as possible.
- Groups can overlap and all overlapping groups must be used.
- If an individual cell cannot be contained in any group it is treated as being a group.
- Within each group, the only input values retained are those which retain a constant value throughout the group.

These rules define a column and a row group as indicated by the blue outlines. In the column group, B remains unchanged but A changes so B is retained. In the row group, it is A that remains unchanged. The Boolean algebra expression is then just the sum of these retained values:

$$X = A + B$$

Thus, the Karnaugh map has found the OR expression without using any algebra.

WORKED EXAMPLE 19.03

Using a K-map to interpret a three-input problem

Let's consider the following truth table:

A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Table 19.07 A sample truth table with three inputs

Before starting any application of a method it is always worth looking to see if there are any trends. In this case you can see that whenever B = 1 the output for X is 1. This means that the final algebra should have B + something.

Applying sum of products gives the following five-minterm expression:

$\overline{A}.\overline{B}.\overline{C} + \overline{A}.B.\overline{C} + \overline{A}.B.C + A.B.\overline{C} + A.B.C$

There are two options for how the K-map is presented. We will choose to combine input values in the columns.

Figure 19.09 shows the result. This follows the convention of having the rows corresponding to values of A and the columns to combinations of values for B and C.

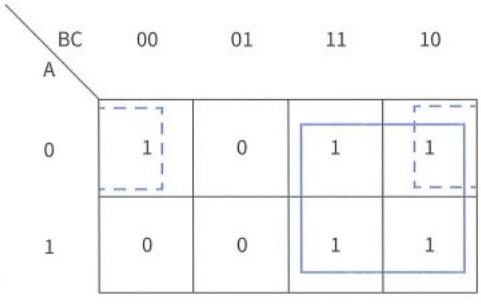


Figure 19.09 A K-map representation of the truth table shown in Table 19.07

(Continued)

It is important to note that the labelling of the columns does not follow a binary value pattern. Instead it follows the Gray coding sequence where only one bit changes value each time.

Following the rules stated above the first group to identify is the square of four cells with a value 1 as identified by the blue rectangle in the diagram. For these it can be seen that A has different values, B has a constant value but C changes values. So, only B is retained. Note this was anticipated from the initial inspection of the truth table.

This apparently only leaves the top left cell. It looks like an isolated cell but it is not because K-maps wrap round. The cell is defined by BC = 00. This has two adjacent cells under Gray coding rules. It is immediately obvious that the cell BC = 01 is adjacent but this contains 0 so it can be ignored. The other adjacent cell is the BC = 10 combination because of the wrap round rule. Thus, there is a row group containing BC = 00 and BC = 10, indicated by the dotted line partial group outlines. For this row the value A remains unchanged, B changes but $\overline{C}$ remains unchanged so the product $\overline{A}.\overline{C}$ results. By adding this to the B for the other group the final expression becomes:

$$\overline{A}.\overline{C} + B$$

This is much simpler than the expression with five minterms derived directly from the truth table.

Extension Question 19.02

Consider the Karnaugh map shown in Figure 19.10. This corresponds to a problem with four inputs. It wraps round horizontally and vertically. Use the map to create a Boolean algebra expression.

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	0	1	1
11	0	0	1	1
10	1	0	0	0

Figure 19.10 A K-map for a four input problem

Reflection Point:

When using Karnaugh maps will you remember to use Gray coding if there are three inputs and will you remember to use all overlapping groups?